# Chapitre 3 : Introduction au C++

**IN323 - Programmation en langage C++**

Elisa HENRION-GUENEAU

IPSA - Site Toulouse

AERO3 - 2025/2026

# Intro

# Introduction

Le C++ est un langage de programmation :

- Créé en 1983
- Inspiré du langage C (avec des fonctionnalités de programmation orientée objet)
- Beaucoup utilisé dans les systèmes embarqués, les jeux vidéos, et les applications hautes performances.
- Puissant, flexible et efficace

# C vs C++

## C :

- Langage impératif
- Moins de vérification de types
- Gestion manuelle complexe de la mémoire
- Bibliothèques standard limitée
- Portable et très performant

## C++ :

- Langage multi-paradigme (impératif et orienté objet)
- Classes et objets → Modularité et réutilisabilité
- Gestion de la mémoire améliorée (new, delete)
- Standard Template Library (STL) + bibliothèque riche pour structures de données et algos
- Portable, performant et adapté aux applications complexes

# Premier programme en C++

```
#include <iostream>
using namespace std;

int main() {
    /* Le programme commence
    son execution ici */
    cout << "Hello world !" << endl;
    return 0; // fin reussie du programme
}
```

**Compilation** : `g++ nomFichier.cpp -o nomExecutable`
**Exécution** : `./nomExecutable`

*La compilation se fait automatiquement avec les IDE comme VS Code, mais elle est indispensable.*

# Explications

- `include <iostream>` : directive de préprocesseur qui inclut la bibliothèque standard d'entrée/sortie (`iostream`) Nécessaire pour utiliser `cout`
- `using namespace std` : permet d'utiliser les éléments de la bibliothèque standard sans avoir besoin d'écrire le préfixe `std::`
- `cout` : objet qui représente la console
- `<<` : opérateur d'insertion *(comme des flèches qui envoient la données vers la console)*
- `endl` : passe à la ligne suivante, et vide le tampon.

# Variables

# Variables et types

En C++, une variable doit être **déclarée avec un type** spécifique avant d'être utilisée.

| Type | Description | Taille |
|---|---|---|
| `bool` | booléen (*true* / *false*) | 1 octet |
| `char` | caractère | 1 octet |
| `int` | entier | 4 octets |
| `long` | (grand) entier | 8 octets |
| `float` | nombre à virgule (simple précision) | 4 octets |
| `double` | nombre à virgule (double précision) | 8 octets |
| `string` | chaîne de caractères (texte) | Variable |
| `void` | ensemble vide de valeurs | N/A |

# Variables et types

```
int age = 30; // Declaration et initialisation
float prix; // Declaration simple
prix = 19.99; // Affectation
char c = 'a';
bool b = false;
string s = "Bonjour"; // ajouter #include <string>
```

# Affichage des variables avec cout

- Utilisation de `std::cout` pour afficher des infos à l'écran

```
#include <iostream>
using namespace std;

int main() {
    int age = 30;
    cout << "L'age est :" << age << endl;
    return 0;
}
```

- Le `std::` est inutile avec `using namespace std`
- « : opérateur d'insertion pour ajouter des données à la sortie

# Affichage des variables avec cout

- Utilisation de `std::cin` pour lire des infos

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Entrez votre age : ";
    cin >> age;
    cout << "Vous avez " << age << " ans" << endl;
    return 0;
}
```

- Le `std::` est inutile avec `using namespace std`
- » : opérateur d'extraction pour capturer les entrées

# Constantes

Il est possible de définir une constante en utilisant `const` ou `constexpr`

```
#include <iostream>
using namespace std;
// Constante globale : connue des la compilation
constexpr double PI = 3.14159;

int main() {
    const double rayon = 10.5; // Constante locale
    double surface = PI * rayon * rayon;
    cout << "Rayon : " << rayon << endl;
    cout << "Surface du cercle : " << surface <<
        endl;
    // rayon = 20.0; // Erreur !
    return 0;
}
```

# Fonctions

# Fonctions : ce qui ne change pas

Le C++ hérite de la structure fondamentale du langage C :

- **Déclaration et Définition :** Distinction entre prototype et corps de la fonction .
- **Syntaxe :** `typeRetour nom(parametres) { ... }`
- **Type `void` :** Utilisé pour les fonctions qui ne renvoient rien.
- **Passage par valeur :** Par défaut, les arguments sont copiés (les modifier dans la fonction ne change pas la variable d'origine).

**Rappel sur les prototypes**

Le compilateur a toujours besoin de connaître la "signature" de la fonction avant son appel pour vérifier la validité des types.

# Fonctions : Les nouveautés du C++

Améliorations pour simplifier le code :

## 1. Le passage par référence (&)

Permet de modifier directement la variable passée en argument **sans utiliser de pointeurs**.

**En C (Pointeur) :**

```
void incr(int *n) {
    *n += 1;
}
incr(&n);
```

**En C++ (Référence) :**

```
void incr(int &n) {
    n += 1;
}
incr(n);
```

- Une référence est un **alias** (un 2nd nom) pour une variable existante.
- Toute modification de la référence modifie directement la variable d'origine.

# Fonctions : Les nouveautés du C++

## 2. La surcharge de fonctions (Overloading)

Il est possible de créer plusieurs fonctions avec le **même nom**, à condition que les paramètres soient différents.

```
int carre(int n);
double carre(double x);
```

## 3. Paramètres par défaut

On peut donner une valeur "de secours" à un paramètre.

```
void log(string msg, int niveau = 1);
// Peut s'appeler : log("Test"); ou log("Test", 3);
```

Attention : les paramètres par défaut doivent toujours être placés à la fin de la liste des paramètres

# Pointeurs

# Pointeurs : ce qui change

Le concept reste identique au C, mais la syntaxe de gestion évolue.

## 1. Plus de NULL mais nullptr

On utilise `nullptr` pour indiquer qu'un pointeur ne pointe sur rien.

```
int* p = nullptr;
```

## 2. Allocation dynamique : new et delete

On n'utilise plus `malloc` et `free`.

```
int* p = new int; // Alloue un entier
*p = 42;
int* q = new int(43);
delete p;         // Libere la memoire
delete q;
```

# Pointeurs vs Références : Lequel choisir ?

## Privilégiez la Référence pour...

- **La sécurité :** ne peut pas être nulle (`nullptr`).
- **La clarté :** se manipule comme une variable normale (pas d'`*`).
- **L'intention :** On l'utilise quand on veut simplement donner un "accès direct" à une variable existante.

## Utilisez un Pointeur uniquement si...

- **La valeur peut être absente :** Si vous avez besoin de représenter l'état "rien" avec `nullptr`.
- **Le changement de cible :** Si vous devez pointer vers une variable A, puis plus tard vers une variable B.
- **L'allocation dynamique :** Pour gérer manuellement la mémoire avec `new`.

# Tableaux

# Les Tableaux Statiques

Le fonctionnement de base reste identique au langage C :

- **Déclaration :** `type nom[taille_fixe];`
- Les éléments sont toujours stockés de manière contiguë.
- L'accès se fait toujours via l'indice `tab[i]`.

```
int t1[5]; // Reserve l'espace pour 5 entiers
int t2[3] = {12, 15, 18};
int t3[] = {1,2,3};
```

# Le lien Tableaux et Pointeurs

En C++, comme en C, le nom d'un tableau est une **adresse constante** vers son premier élément.

- `tab` est strictement équivalent à `&tab[0]`.
- L'accès `tab[i]` est une écriture simplifiée de `*(tab+i)`.

```
int tab[3] = {10, 20, 30};
int* p = tab; // p pointe sur le 1er element (10)
cout << tab[1] << endl;   // Acces classique : 20
cout << *(p+1) << endl; // Acces via pointeur : 20
```

# Les chaînes de caractères

En C++, on n'utilise plus (ou très peu) les tableaux de `char`.
On utilise l'objet **`string`**.

```
#include <iostream>
#include <string>
using namespace std;

string nom = "C";
nom += "++"; // Concatenation directe
if (nom == "C++") {
    cout << "Texte identique";
}
```

**Avantages de `std::string` :**

- **Dynamique :** La taille s'adapte toute seule.
- **Opérateurs :** +, =, == fonctionnent normalement.
- **Sécurité :** Plus besoin de gérer le `'\0'` final.

# Tableaux Dynamiques : new et delete[]

En C++, on remplace malloc et free par des **opérateurs** natifs plus simples.

```
int n;
cout << "Nombre d'elements : ";
cin >> n;

// ETAPE 1 : Allocation (plus besoin de sizeof)
int *tab = new int[n];

// ... ETAPE 2 : Utilisation classique (tab[i]) ...

// ETAPE 3 : Liberation de la memoire
delete[] tab; // les crochets pour un tableau
```

**Sécurité :** Si l'allocation échoue, le C++ lève une exception (ou renvoie nullptr).

# Structures

# Les Structures : La base

Comme en C, une `struct` permet de regrouper des données de types différents sous un même nom.

```
struct Personne {
    string nom;
    int age;
};
int main() {
    Personne p1; // mot-cle 'struct' facultatif
    p1.nom = "Alice";
    p1.age = 20;
    Personne p2 = {"Bob", 22};
    return 0;
}
```

Plus besoin de faire un `typedef` pour éviter d'écrire `struct Personne` partout.

# Structures : Pointeurs et Références

Comment manipuler une structure sans la copier ?

```
void modifierAgePtr(Personne* p) {
    p->age += 1;     // fleche (Pointeur)
}
void modifierAgeRef(Personne &p) {
    p.age += 1;      // point (Reference)
}
int main() {
    Personne alice = {"Alice", 20};
    modifierAgePtr(&alice); // On passe l'adresse
    modifierAgeRef(alice);  // On passe l'objet
    return 0;
}
```

# Allocation dynamique de structure

Le C++ simplifie la création d'objets sur le Tas (Heap).

**En C**

```
Personne *p = malloc(
    sizeof(Personne));
strcpy(p->nom, "Bob");
p->age = 22;
...
free(p);
```

**En C++**

```
Personne *p = new
    Personne{"Bob", 22};
...
delete p;
```

- **new** : Alloue la mémoire et initialise les données d'un seul coup.
- Plus besoin de `sizeof` ou de `strcpy`.

# Biblio

# Mathématiques et Caractères

Le C++ utilise les bibliothèques du C sous un nouveau nom.

## Mathématiques : `<cmath>`

Plus besoin de différencier `abs` et `fabs`

- `abs(x)` : valeur absolue (marche pour `int` et `double`).
- `pow(x, y)`, `sqrt(x)`, `round(x)`, `cos(x)`...

## Analyse de caractères : `<cctype>`

Identique au C, très utile pour valider des saisies utilisateur :

- `isdigit(c)`, `isalpha(c)`, `islower(c)`...
- `toupper(c)`, `tolower(c)`

*Note : Toutes ces fonctions sont dans l'espace de nom `std::`*

# Temps et Aléatoire

On utilise <ctime> et <cstdlib>. La logique reste la même qu'en C.

```
#include <ctime>
#include <cstdlib>

int main() {
    // Initialisation de la graine sur le temps
        actuel
    srand(time(0));
    int n = rand() % 100; // nombre entre 0 et 99
    cout << "Nombre aleatoire : " << n << endl;
    cout << "Secondes depuis 1970 : " << time(0) <<
        endl;
    return 0;
}
```